

Struktur einfacher Webanwendungen

Spengergasse

3. März 2026

Einleitung

- Dieser Foliensatz zeigt den strukturierten Aufbau einer einfachen Webanwendung am Beispiel einer Todo-App.
- Die Anwendung wird in acht Bausteine zerlegt: Zustand, Zustandszugriff, DOM-Referenzen, DOM-Erzeugung, Rendering, Event-Handling, Bindings und Initialisierung.
- Ziel ist ein klarer Ablauf: Benutzerinteraktion → Zustandsänderung → Neurendern der Oberfläche.
- Die folgenden Detailfolien zeigen dazu den konkreten JavaScript-Code pro Baustein.

Struktur einfacher WebApps

- 1 Application State
- 2 State Accessors/Mutators
- 3 DOM Node References
- 4 DOM Node Creation Functions
- 5 Render Functions
- 6 Event Handlers
- 7 Initialization of Bindings
- 8 Initial Rendering

- ❶ Anwendungszustand
 - Enthält den Zustand der Anwendung
 - Eine zentrale Quelle der Wahrheit für den Anwendungszustand
- ❷ State-Accessor-/Mutator-Funktionen
 - Funktionen zum Lesen und Ändern des Zustands
 - Hier verwendet zum Hinzufügen, Umschalten und Löschen von Todos
- ❸ DOM-Knoten-Referenzen
 - Statische Referenzen auf DOM-Knoten, die nach dem Start benötigt werden
- ❹ Funktionen zur DOM-Knoten-Erzeugung
 - Dynamische Erzeugung von DOM-Knoten bei Benutzerinteraktionen
 - Hier verwendet, um ein Todo-Listenelement zu erzeugen
- ❺ Render-Funktionen
 - Rendern den Anwendungszustand ins DOM
 - Einfaches vollständiges Neurendern beider Listen
 - Der Zustand steuert die UI: Jede Zustandsänderung triggert ein Neurendern
- ❻ Event-Handler
 - Behandeln Benutzerinteraktionen (Klick/Änderung)
 - Rufen State-Mutatoren auf und rendern danach
 - Namenskonvention: `on<Event>`
- ❼ Initialisierung der Bindings
 - Initiales Verbinden der Event-Handler
- ❽ Initiales Rendern
 - Initialer Aufruf von `render()`, um den Startzustand anzuzeigen

HTML Struktur

```
1  <body>
2    <h1>Todo List</h1>
3    <input id="todo-input" type="text" placeholder="What needs to be
4    <button id="todo-add" type="button">Add</button>
5
6    <h2>Pending Tasks</h2>
7    <ul id="todo-list">
8      <!-- dynamic content goes here -->
9      <!--<li>
10         <input type="checkbox" checked>
11         CABS lernen
12         <button>remove</button>
13       </li>
14     -->
15   </ul>
16
17   <h2>Done</h2>
18   <ul id="todo-list-done">
19     <!-- dynamic content goes here -->
20   </ul>
21 </body>
```

1) Application State

Der Anwendungszustand enthält die Daten der App und ist die zentrale Quelle der Wahrheit.

```
1  state = {  
2    todos: [  
3      { text: 'CABS□lernen', completed: false },  
4      { text: 'NSCS□lernen', completed: true },  
5      { text: 'POS/Theorie□lernen', completed: false },  
6      { text: 'WMC□programmieren', completed: false },  
7    ],  
8  };
```

2) State Accessors/Mutators

Diese Funktionen lesen und ändern den Zustand; hier werden Todos hinzugefügt und umgeschaltet.

```
1  function addNewTodoItemToState(text) {
2      state.todos.push({ text: text, completed: false });
3  }
4
5  function toggleTodoState(todoObject) {
6      todoObject.completed = !todoObject.completed;
7  }
```

2) State Accessors/Mutators (2/2)

```
1  function deleteTodoObjectFromState(todoObject) {  
2      let idx = state.todos.indexOf(todoObject);  
3      if (idx !== -1) {  
4          state.todos.splice(idx, 1);  
5      }  
6  }
```

3) DOM Node References

Statische Referenzen auf DOM-Knoten, die nach dem Start der Anwendung mehrfach benötigt werden.

```
1  const todoList = document.getElementById('todo-list');
2  const todoListDone = document.getElementById('todo-list-done');
3  const addBtn = document.getElementById('todo-add');
4  const newTodoText = document.getElementById('todo-input');
```

4) DOM Node Creation Functions

DOM-Knoten werden dynamisch bei Benutzerinteraktionen erzeugt; hier wird ein Todo-Listenelement gebaut.

```
1 function createTodoElement(todo) {
2   const listItem = document.createElement('li');
3   listItem.textContent = todo.text;
4
5   const cb = document.createElement('input');
6   cb.type = 'checkbox';
7   cb.checked = todo.completed;
8
9   cb.addEventListener('change', (event) => onTodoStateChanged(todo))
10
11  const delBtn = document.createElement('button');
12  delBtn.textContent = 'delete';
13  delBtn.addEventListener('click', (event) => onTodoObjectDelete(todo))
14
15  listItem.prepend(cb);
16  listItem.append(delBtn);
17
18  return listItem;
19 }
```

5) Render Functions

Render-Funktionen übertragen den Zustand ins DOM. Hier wird einfach vollständig neu gerendert; jede Zustandsänderung triggert ein Re-Render.

```
1  function render() {  
2      todoListDone.innerHTML = '';  
3      for (let todoItem of state.todos) {  
4          if (todoItem.completed) {  
5              const li = createTodoElement(todoItem);  
6              todoListDone.appendChild(li);  
7          }  
8      }  
9  }
```

6) Event Handlers

Event-Handler verarbeiten Benutzeraktionen, rufen State-Mutatoren auf und rendern anschließend neu (Namensschema: on<Event>).

```
1  function onAddBtnPressed() {  
2    // alert('button pressed: ' + newTodoText.value);  
3    addNewTodoItemToState(newTodoText.value);  
4    render();  
5  }
```

6) Event Handlers (2/2)

```
1  function onTodoStateChanged(todoObject) {
2      toggleTodoState(todoObject);
3      render();
4  }
5
6  function onTodoObjectDelete(todoObject) {
7      deleteTodoObjectFromState(todoObject);
8      render();
9  }
```

7) Initialization of Bindings

Hier werden die Event-Handler initial mit den DOM-Elementen verbunden.

```
1      addBtn.addEventListener('click', (event) => onAddBtnPressed());
```

8) Initial Rendering

Zum Start wird `render()` einmal aufgerufen, um den Anfangszustand anzuzeigen.

```
1  render();
```